A project on

"Face Recognition Attendance System"

Submitted by

Shivam Tripathi

B.Sc. Part III

Enrollment Number U2311071

Roll Number 2679463

Under the supervision of

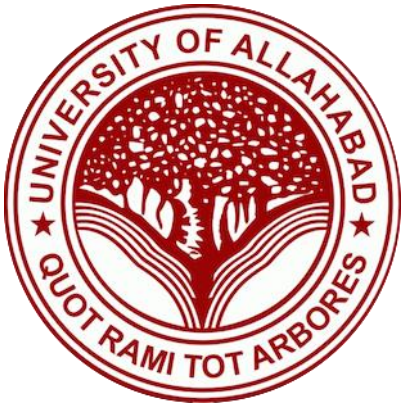
Dr. Nimish Kumar Srivastava

Department Electronics and Communication (JKIAPT)

Faculty of Science

University of Allahabad

Prayagraj, 211002, India



Acknowledgement

I would like to thank my supervisor **Dr. Nimish Kumar Srivastava** for guiding me throughout this project. Their support and advice helped me complete this work successfully.

I also thank the Head of CS Department and all my friends who provided resources and guidance during the development of this project.

I am grateful to my friends and family for their encouragement and support throughout my academic journey.

Shivam Tripathi

BSc Part III

Department Electronics and Communication (JKIAPT)

University of Allahabad

Preface

Taking attendance using pen and paper is slow and can have mistakes. Teachers waste time calling names, and students sometimes mark attendance for friends who are absent. There is a need for a better and automatic system.

This project, Face Recognition Attendance System (Tinter), is a desktop application built using Python. It uses the camera to see the face of a student and marks their attendance automatically. No manual work is needed. The student just stands in front of the camera and the system does everything.

The system has two parts. The first part is for Admin, who can add students, take their face photos, and train the system. The second part is for Students, who just use the camera to mark attendance. All records are saved in a file and can be searched, sorted, and exported.

This project shows how computer vision and Python programming can solve a real-world problem in a simple and practical way.

Chapter 1: Introduction 4
1.1 Introduction....5
1.2 Present Scenario....5
1.3 Objective of the Work....5
1.4 Scope of the Project....6
Chapter 2: Background Study....6
2.1 Face Recognition Technology....7
2.2 OpenCV and LBPH Algorithm....7
2.3 Existing Attendance Systems....7
2.4 Need for Automated Attendance....8
Chapter 3: Proposed System....8
3.1 Introduction of the Work....9
3.2 Methodology....9
3.3 System Architecture and Flow....9
3.4 Implementation....10
3.4.1 database.py — Database Module....10
3.4.2 face_system.py — Face Recognition Engine....11
3.4.3 main.py — GUI Module....14
3.5 Test Cases....15
3.5.1 Test Case 1 — Successful Attendance....16
3.5.2 Test Case 2 — Already Marked Today....16
3.5.3 Test Case 3 — Unknown Face....16
3.5.4 Test Case 4 — Admin Deletes Attendance Record....16
Chapter 4: Results and Analysis....17
4.1 Results....18
4.2 Discussion....18
4.3 Limitations....18
4.4 Future Improvements....19
References....19

Chapter 1: Introduction

1.1 Introduction

Attendance is very important in schools and colleges. But taking attendance by calling names or using a register takes a lot of time. It can also have errors. Sometimes students mark attendance for their absent friends, which is called proxy attendance. This is a big problem in many institutions.

This project, Tinter, solves this problem using face recognition. The system uses the webcam of a computer to detect a student's face. When the face is recognized, the system automatically writes the student's name, date, and time in a file. The student does not need to do anything — just stand in front of the camera.

The project is made using Python language. It uses OpenCV for face detection and recognition. The screen and buttons are made using Tkinter. Student information is stored in an SQLite database. Attendance records are saved in a CSV file which can be opened in Excel.

Admin can log into the system and add new students, capture their face photos, and train the system. After training, the system can recognize all registered faces automatically. Admin can also see, search, sort, export, and delete attendance records.

1.2 Present Scenario

In most colleges today, attendance is still taken manually. A teacher calls names one by one or passes a register. This takes 5 to 10 minutes of every class. If there are many students, it takes even more time.

Another big problem is proxy attendance. A student who is absent asks a friend to mark their name in the register. Because of this, the attendance record is not correct. Teachers cannot trust it fully.

Some places use fingerprint machines, but these need physical touch. During COVID-19, touching a shared machine was not safe. A contactless system is much better.

Face recognition is the best solution because it is contactless, automatic, and fast. The student just walks in front of the camera. The system does everything in a few seconds. No register, no calling names, no manual work.

1.3 Objective of the Work

The main objectives of this project are:

- To build an automatic attendance system that works using the webcam and face recognition.
- To use the LBPH algorithm from OpenCV which works well on a normal computer without any special hardware.
- To make a simple and easy-to-use graphical interface using Tkinter so that any person can use it easily.
- To allow admin to register students, capture face images, and train the model from inside the application.
- To automatically mark attendance when a registered face is seen in the camera.
- To show green color when attendance is marked, yellow when already marked today, and red when face is not recognized.
- To allow admin to search, sort, export, and delete attendance records easily.
- To store all student data in an SQLite database and attendance in a CSV file.

1.4 Scope of the Project

This project is made for educational institutions like schools and colleges. It can also be used in small offices. The system works on any Windows computer with a webcam. It does not need internet or any cloud service. Everything runs on the local computer.

The system can handle multiple students. It marks attendance only once per day for each student. If a student tries again on the same day, a yellow warning is shown and no duplicate entry is made.

This project is mainly for academic learning. It shows how face recognition can be used to solve a real problem. It does not do liveness detection, which means it cannot tell if someone is using a photo instead of a real face. This can be improved in the future.

Chapter 2: Background Study

2.1 Face Recognition Technology

Face recognition is a technology that finds and identifies a person by looking at their face. It works in four steps. First, it finds the face in the image (detection). Second, it makes the face image ready for comparison (alignment). Third, it converts the face into numbers (feature extraction). Fourth, it compares those numbers with saved records to find a match.

Face recognition is used in many places today. Phones use it to unlock the screen. Airports use it for security. Social media uses it to tag people in photos. In this project, it is used to mark student attendance automatically.

The biggest advantage of face recognition is that it is contactless. The person does not need to touch anything. They just look at the camera. This makes it hygienic and fast.

2.2 OpenCV and LBPH Algorithm

OpenCV (Open Source Computer Vision Library) is a free library for image and video processing. It is written in C++ but can be used in Python easily. OpenCV provides tools for detecting faces, recognizing people, reading webcam video, and much more.

This project uses two things from OpenCV:

- Haar Cascade Classifier: This is used to detect faces in each frame of the video. It is trained to find frontal faces. It is fast and works in real time.
- LBPH Face Recognizer: LBPH stands for Local Binary Pattern Histogram. It divides the face image into small blocks, looks at each pixel and its neighbors, converts the result into a binary number, and then makes a histogram. This histogram is saved during training. During recognition, the live face histogram is compared with saved ones to find the closest match.

LBPH is good for this project because it works well in different lighting conditions and does not need a lot of training images. Only 30 to 50 photos per person are enough. It also runs fast on a normal computer.

2.3 Existing Attendance Systems

There are many types of attendance systems already being used:

- Manual Register: Teacher calls names and writes in a register. It is slow and allows proxy attendance.

- RFID Card System: Each student gets a card. The card is scanned at entry. Cards can be lost or shared, so it is not fully reliable.
- Fingerprint System: Uses a fingerprint scanner. It is accurate but requires physical touch, which is not good in hygienic environments.
- Iris Recognition: Very accurate but needs very expensive hardware. Not practical for small institutions.
- Face Recognition System: Contactless, fast, and automatic. Works with a normal webcam. This is what Tinter uses.

2.4 Need for Automated Attendance

There are clear reasons why an automated face recognition attendance system is needed:

- Proxy attendance is a serious problem. Face recognition makes it impossible because only the real face is accepted.
- Manual attendance wastes class time. An automated system marks attendance in seconds.
- Paper records are hard to manage and search. Digital records in a CSV file are easy to view, export, and analyze.
- After COVID-19, contactless systems are preferred for hygiene and safety.
- Human errors in manual entry can be avoided with automatic recording.

Chapter 3: Proposed System

3.1 Introduction of the Work

Tinter is a Python desktop application that automatically marks attendance using face recognition. It has three main Python files:

- database.py: Handles all student data using SQLite.
- face_system.py: Handles all face related work — capturing images, training the model, and recognizing faces.
- main.py: Shows the graphical interface using Tkinter.

The name Tinter comes from two words: Tkinter (the GUI library) and the face recognition engine. The system works completely offline. All data is stored on the local computer.

3.2 Methodology

The system works in the following steps:

1. Admin adds a new student by entering name, gender, course, and section. This is saved in the database.
2. Admin clicks the Capture Face button. The webcam opens and takes 50 face photos of the student.
3. After capturing, the system trains the LBPH model using all collected photos. The model is saved as trainer.yml.
4. When a student clicks the Attendance button, the webcam opens inside the Tkinter window.
5. The system reads each frame, detects the face, and predicts the student ID using the trained model.
6. If the confidence score is below 60, the face is accepted and attendance is recorded in attendance.csv.
7. Admin can view all attendance records, search by name or date, sort, export to CSV, or delete selected records.

3.3 System Architecture and Flow

The system is divided into three modules:

Module	File	What it does
Database	database.py	Save, read, delete student records in SQLite
Face Engine	face_system.py	Capture photos, train model, recognize face, mark attendance
GUI	main.py	Home screen, admin panel, camera window, help page

System flow:

- Home Screen has three buttons: Attendance, Admin Login, and Help.
- Clicking Attendance opens the camera inside Tkinter. Face is detected and recognized. Attendance is marked automatically. Camera closes after 2 seconds.
- Admin Login opens the admin panel (Username: admin, Password: 1234).
- Admin Panel has Add Student, Show Students, and Attendance Data buttons.

3.4 Implementation

3.4.1 database.py — Database Module

This file manages all student data using Python's built-in sqlite3 library. It connects to a file called attendance.db and performs all database operations.

connect_db() function:

This function opens a connection to the database file. Every other function calls this first.

```
def connect_db():
    return sqlite3.connect("attendance.db")
```

create_tables() function:

This function creates the students table when the program runs for the first time. It also adds missing columns automatically if they were not there before.

```
def create_tables():
```

```
    conn = connect_db()
    cur = conn.cursor()

    cur.execute("""
    CREATE TABLE IF NOT EXISTS students(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT
    )
    """)

    try:
        cur.execute("ALTER TABLE students ADD COLUMN gender TEXT")
    except:
        pass

    try:
        cur.execute("ALTER TABLE students ADD COLUMN course TEXT")
    except:
        pass

    try:
        cur.execute("ALTER TABLE students ADD COLUMN section TEXT")
    except:
        pass

    conn.commit()
    conn.close()
```

add_student() function:

This function adds a new student to the database and returns the auto-generated student ID. This ID is later used to name the face image files.

```
def add_student(name, gender, course, section):
    conn = connect_db()
    cur = conn.cursor()
    cur.execute(
        "INSERT INTO students(name,gender,course,section) VALUES(?,?,?,?)",
        (name, gender, course, section)
    )
    conn.commit()
    sid = cur.lastrowid
    conn.close()
    return sid
```

get_name() function:

This function finds the student name from the database using their ID. It is called during attendance marking to write the correct name into the CSV file.

```
def get_name(sid):
    conn = connect_db()
    cur = conn.cursor()
    cur.execute("SELECT name FROM students WHERE id=?", (sid,))
    data = cur.fetchone()
    conn.close()
    return data[0] if data else "Unknown"
```

3.4.2 face_system.py — Face Recognition Engine

This file handles all face-related work using OpenCV. It has four main functions.

capture_images() function:

This function opens the webcam and captures 50 grayscale face photos of a student. Each photo is saved in the dataset folder with the student ID in the filename (e.g., User.3.1.jpg). The Haar Cascade classifier is used to detect the face region in each video frame.

```
def capture_images(sid):
    cam = cv2.VideoCapture(0)
    detector = cv2.CascadeClassifier(
        cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
    )

    if not os.path.exists("dataset"):
        os.makedirs("dataset")

    count = 0

    while True:
        ret, img = cam.read()
        if not ret:
            break

        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        faces = detector.detectMultiScale(gray, 1.3, 5)

        for (x, y, w, h) in faces:
            count += 1
            cv2.imwrite(f"dataset/User.{sid}.{count}.jpg",
                        gray[y:y+h, x:x+w])
            cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)

        cv2.imshow("Capturing Face", img)

        if cv2.waitKey(1) == 13 or count >= 50:
            break

    cam.release()
    cv2.destroyAllWindows()
```

train_model() function:

This function reads all face images from the dataset folder, extracts the student ID from each filename, and trains the LBPH recognizer. The trained model is saved as trainer.yml and is used later for recognition.

```
def train_model():
    path = 'dataset'
    recognizer = cv2.face.LBPHFaceRecognizer_create()

    faces = []
    ids = []

    for file in os.listdir(path):
        img_path = os.path.join(path, file)
        img = Image.open(img_path).convert('L')
        img_np = np.array(img, 'uint8')

        sid = int(file.split(".")[1])  # extract ID from filename

        faces.append(img_np)
        ids.append(sid)

    recognizer.train(faces, np.array(ids))
    recognizer.save('trainer.yml')
```

mark_attendance() function:

This function checks if the student has already been marked today. If not, it writes their ID, name, date, and time into the attendance.csv file. It returns 'marked' if attendance was recorded, or 'already' if it was already done today.

```
def mark_attendance(sid):
    name = get_name(sid)
    now  = datetime.now()
    date = now.strftime("%Y-%m-%d")
    time = now.strftime("%H:%M:%S")

    file_exists = os.path.isfile("attendance.csv")
    already = False

    if file_exists:
        with open("attendance.csv", "r") as f:
            for row in csv.reader(f):
                if len(row) >= 3 and row[0] == str(sid) and row[2] == date:
                    already = True
                    break

    if not already:
        with open("attendance.csv", "a", newline="") as f:
            writer = csv.writer(f)
            if not file_exists:
                writer.writerow(["ID", "Name", "Date", "Time"])
            writer.writerow([sid, name, date, time])

        winsound.Beep(1000, 300)
        return "marked"
```

```
    return "already"
```

recognize_face() function:

This is the main function for attendance. It loads the trained model, opens the webcam, detects faces in each frame, and predicts the student ID. If the confidence score is below 60, the face is accepted. The mark_attendance() function is then called and the result is shown on screen with color coding.

```
def recognize_face():
    recognizer = cv2.face.LBPHFaceRecognizer_create()
    recognizer.read('trainer.yml')

    detector = cv2.CascadeClassifier(
        cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
    )

    cam = cv2.VideoCapture(0)
    close_after = None

    while True:
        ret, img = cam.read()
        if not ret:
            break

        gray  = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        faces = detector.detectMultiScale(gray, 1.2, 5)

        for (x, y, w, h) in faces:
            sid, conf = recognizer.predict(gray[y:y+h, x:x+w])

            if conf < 60:           # confident match
                name   = get_name(sid)
                result = mark_attendance(sid)

                if result == "marked":
                    color = (0, 255, 0)   # green
                    close_after = datetime.now()
                elif result == "already":
                    color = (0, 200, 255) # yellow
            else:
                name  = "Unknown"
                color = (0, 0, 255)       # red

            cv2.putText(img, name, (x, y-10), 2, 1, color, 2)
            cv2.rectangle(img, (x, y), (x+w, y+h), color, 2)

        cv2.imshow("Face Recognition", img)

        if cv2.waitKey(1) == 13:
            break

        # auto-close 2 seconds after attendance is marked
```

```
        if close_after is not None:
            if (datetime.now() - close_after).total_seconds() >= 2:
                break

    cam.release()
    cv2.destroyAllWindows()
```

3.4.3 main.py — GUI Module

This file builds the complete graphical interface using Tkinter. It connects the database module and the face system module together.

Home Screen:

The home() function clears the screen, sets a background image, and shows three buttons: Attendance, Admin Login, and Help.

```
def home():
    clear()
    set_bg()
    title("FACE RECOGNITION ATTENDANCE SYSTEM")

    img_btn("images/attendance.jpg", "Attendance", 200, 200, recognize_face)
    img_btn("images/admin.jpg",      "Admin Login", 380, 200, admin_login)
    img_btn("images/help.jpg",       "Help",        560, 200, help_page)
```

Admin Login:

The admin_login() function opens a small popup window with a username and password field. If the credentials are correct (admin / 1234), the admin panel opens.

```
def admin_login():
    win = tk.Toplevel(root)

    tk.Label(win, text="Username").pack()
    u = tk.Entry(win); u.pack()

    tk.Label(win, text="Password").pack()
    p = tk.Entry(win, show="*"); p.pack()

    def login():
        if u.get() == "admin" and p.get() == "1234":
            win.destroy()
            admin_panel()
        else:
            messagebox.showerror("Error", "Wrong login")

    tk.Button(win, text="Login", command=login).pack()
```

Add Student:

The add_student_ui() function shows a form with four fields: Name, Gender, Course, and Section. When Save Data is clicked, the student is added to the database. When Capture Face is clicked, the camera opens and face images are collected and the model is trained.

```
def add_student_ui():
    win = tk.Toplevel(root)

    name    = tk.Entry(win); name.grid(row=0, column=1)
    gender  = tk.Entry(win); gender.grid(row=1, column=1)
    course  = tk.Entry(win); course.grid(row=2, column=1)
    section = tk.Entry(win); section.grid(row=3, column=1)

    sid_holder = {"id": None}

    def save():
        sid = add_student(name.get(), gender.get(),
                          course.get(), section.get())
        sid_holder["id"] = sid
        messagebox.showinfo("Saved", "Now Capture Face")

    def capture():
        if sid_holder["id"] is None:
            messagebox.showerror("Error", "Save first")
            return
        capture_images(sid_holder["id"])
        train_model()
        messagebox.showinfo("Done", "Face Added")

    tk.Button(win, text="Save Data",    command=save).grid(row=4, column=0)
    tk.Button(win, text="Capture Face", command=capture).grid(row=4, column=1)
```

3.5 Test Cases

3.5.1 Test Case 1 — Successful Attendance

Scenario: A registered student stands in front of the camera.

Step	What happens
Click Attendance	Camera window opens inside Tkinter
Face Detected	Green box appears around the face
Confidence < 60	Student name shown, record saved in attendance.csv
Status bar	Shows: Attendance Marked: [Name] — Closing...
Auto close	Camera window closes after 2 seconds

3.5.2 Test Case 2 — Already Marked Today

Scenario: A student tries to mark attendance again on the same day.

Step	What happens
Face Recognized	Student ID matched in database
Already in CSV	mark_attendance() returns 'already'
Camera Feed	Yellow box with message: Already Marked Today: [Name]
Action	Camera stays open. No duplicate record is created.

3.5.3 Test Case 3 — Unknown Face

Scenario: An unregistered person stands in front of the camera.

Step	What happens
Face Detected	Face found by Haar Cascade
Confidence >= 60	No confident match found
Camera Feed	Red box shown with label: Unknown
Action	No attendance marked. Camera stays open.

3.5.4 Test Case 4 — Admin Deletes Attendance Record

Scenario: Admin wants to remove a wrong attendance entry.

Step	What happens
Open Attendance Data	Table shows all records from attendance.csv
Select Row	Admin clicks a row to select it

Click Delete	Confirmation dialog shows name and date
Confirm Yes	Record is removed from CSV. Table refreshes.

Chapter 4: Results and Analysis

4.1 Results

The Face Recognition Attendance System (Tinter) was built and tested successfully. The system can recognize registered student faces in real time using a normal laptop webcam. Attendance is marked automatically without any manual work.

The system gives three types of results on the camera screen:

- Green box (Attendance Marked): The face is recognized with a confidence score below 60. Attendance is recorded in attendance.csv. The camera closes automatically after 2 seconds.
- Yellow box (Already Marked): The face is recognized but attendance for today is already in the file. A warning is shown. No duplicate record is created.
- Red box (Unknown): The system cannot match the face to any registered student. The word Unknown is shown. No attendance is marked.

The admin panel worked correctly for all features: adding students, capturing face images, training the model, and managing attendance records with search, sort, export, and delete.

4.2 Discussion

The LBPH algorithm worked well for this project. It is light and fast. It runs in real time on a normal computer without any GPU or deep learning tools. The recognition accuracy was good when face photos were taken in clear lighting and the student was facing the camera directly.

One important design choice was to show the camera feed inside the Tkinter window instead of using a separate OpenCV window. An OpenCV window sometimes goes behind the main application on Windows. By showing the camera on a Tkinter Canvas and updating it with root.after(), the camera window always stays in front and works smoothly with the GUI.

The confidence threshold of 60 gives a good balance. If the threshold is too high, unknown faces may be wrongly accepted. If it is too low, registered students may be rejected. A value of 60 was found to work well during testing.

Saving attendance in a CSV file makes it easy for admin to open the file in Excel, print it, or share it. The database keeps student information organized and easy to search.

4.3 Limitations

The following limitations were found during development and testing:

- Lighting Sensitivity: If the room lighting is very poor or there is strong light behind the student, recognition accuracy can reduce.
- Single Camera Only: The system supports only one webcam at a time. Multi-entry points are not supported.
- Windows Only: The winsound library used for the beep sound works only on Windows. It will not work on Linux or macOS without changes.
- Hardcoded Password: The admin password (1234) is written directly in the code. This is not safe for real deployment.
- No Liveness Detection: The system cannot tell if someone is holding a printed photo in front of the camera. A real face and a photo both look the same to the system.
- No Internet or Cloud: All data is stored locally. There is no backup or remote access.

4.4 Future Improvements

The following improvements can be added to make the system better:

- Liveness Detection: Add blink detection or depth sensing to make sure only a real face is accepted, not a printed photo.
- Better Recognition Model: Replace LBPH with a deep learning model like FaceNet or DeepFace for higher accuracy in difficult conditions.
- Cross-Platform Support: Replace winsound with a library like playsound or pygame to make the system work on Linux and macOS also.
- Secure Admin Login: Store the admin password as a hashed value in the database instead of writing it directly in the code.
- Cloud Backup: Connect the system to cloud storage so attendance records are backed up automatically and can be accessed from anywhere.
- Multi-Camera Support: Allow multiple cameras so students can be recognized at different entry points at the same time.
- Email Reports: Send a daily attendance summary to students or teachers by email automatically.
- Web Interface: Build a web-based admin dashboard so attendance records can be managed from any device through a browser.

References

OpenCV Documentation — https://docs.opencv.org

Python Official Documentation — https://docs.python.org

Tkinter Documentation — https://docs.python.org/3/library/tkinter.html

SQLite Documentation — https://www.sqlite.org/docs.html

Bradski, G. and Kaehler, A. (2008). Learning OpenCV. O'Reilly Media.

Pillow (PIL Fork) Documentation — https://pillow.readthedocs.io